



В этом занятии нам предстоит потренироваться построению нейронных сетей с помощью библиотеки Pytorch. Делать мы это будем на нескольких датасетах.

Для каждого датасета мы будем реализовывать стандартную логику обучения моделей:

1. Подготовим данные
2. Создадим модель (напишем класс модели)
3. Создадим функции обучения на тренировочных данных и применения модели к тестовым данным
4. Проведем обучение, оценим результаты работы моделей

Начнем с импорта необходимых библиотек

```
import numpy as np

import seaborn as sns
from matplotlib import pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

import torch
from torch import nn
from torch.nn import functional as F

from torch.utils.data import TensorDataset, DataLoader
import random
from tqdm.notebook import tqdm

import copy
```

```
sns.set_theme(style="darkgrid", font_scale=1.4)

# Reduce stochasticity

SEED = 42

# PyTorch
torch.manual_seed(SEED)

# CUDA (GPU)
torch.cuda.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED) # For multi-GPU setups

# NumPy
np.random.seed(SEED)

# Python's built-in random
random.seed(SEED)

torch.backends.cudnn.deterministic = True
torch.use_deterministic_algorithms(True)

device = "cuda" if torch.cuda.is_available() else "cpu"

print(device)

cuda
```

Часть 1. Датасет moons

Часть 1.1. Подготовка данных

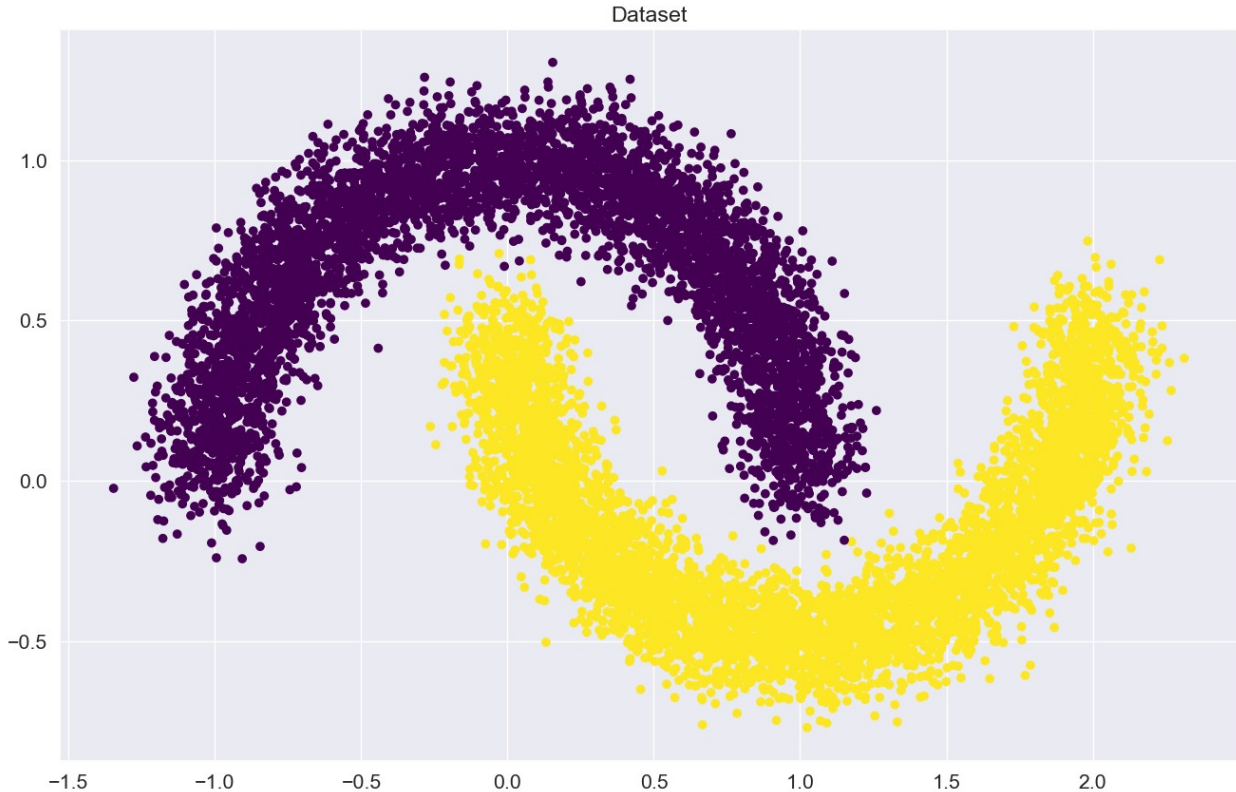
Давайте сгенерируем датасет и посмотрим на него!

```
X, y = make_moons(n_samples=10000, random_state=42, noise=0.1)

# Визуализация данных

plt.figure(figsize=(16, 10))
plt.title("Dataset")
plt.scatter(X[:, 0], X[:, 1], c=y, cmap="viridis")

plt.show();
```



Сделаем train/test split

```
X_train, X_val, y_train, y_val = train_test_split(X, y,  
random_state=42)
```

Задание. Загрузчики данных

В PyTorch загрузка данных как правило происходит налету (иногда датасеты не помещаются в оперативную память). Для этого используются две сущности `Dataset` и `DataLoader`.

1. `Dataset` загружает каждый объект по отдельности.
2. `DataLoader` группирует объекты из `Dataset` в батчи.

Так как наш датасет достаточно маленький мы будем использовать `TensorDataset`. Все, что нам нужно, это перевести из массива numpy в тензор с типом `torch.float32`.

Создайте тензоры с обучающими и тестовыми данными

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)  
y_train_t = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)  
X_val_t = torch.tensor(X_val, dtype=torch.float32)  
y_val_t = torch.tensor(y_val, dtype=torch.float32).reshape(-1, 1)
```

Создаем `Dataset` и `DataLoader`.

Подумайте, нужно ли использовать `shuffle` для:

- train data
- validation data

Почему вы приняли такое решение?

Ответ:

- train_dataloader с параметром shuffle=True для исключения запоминания порядка и лучшего обобщения, что влияет на обучение. Перемешивания в тренировочном лоадере нужны в датасетах, где не важен порядок следования объектов.
- val_dataloader с параметром shuffle=False для воспроизводимости результатов для дальнейшего сравнения и анализа модели.

```
train_dataset = TensorDataset(X_train_t, y_train_t)
val_dataset = TensorDataset(X_val_t, y_val_t)
train_dataloader = DataLoader(train_dataset, batch_size=128,
                              shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=128,
                             shuffle=False)
```

Часть 1.2. Класс модели

Напоминание. Давайте вспомим, что происходит в логистической регрессии. На входе у нас есть матрица объект-признак X и столбец-вектор y – метки из $\{0, 1\}$ для каждого объекта. Мы хотим найти такую матрицу весов W и смещение b (bias), что наша модель $XW + b$ будет каким-то образом предсказывать класс объекта. Как видно, на выходе наша модель может выдавать число в интервале от $(-\infty; \infty)$. Этот выход как правило называют "логитами" (logits). Нам необходимо перевести его на интервал от $[0; 1]$ для того, чтобы он выдавал нам вероятность принадлежности объекта к классу 1, также лучше, чтобы эта функция была монотонной, быстро считалась, имела производную и на $-\infty$ имела значение 0, а на $+\infty$ имела значение 1. Такой класс функций называется сигмоидой. Чаще всего в качестве сигмоиды берут:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Модель логистической регрессии - это по сути **нейронная сеть** с одним линейным слоем и функцией активации сигмоида. Обученная модель возвращает прогнозы вероятностей классов для каждого объекта, по которым можно рассчитать наиболее вероятный класс для каждого объекта.

В первой части задания мы на этой знакомой нам модели потренируем технику работы с нейронными сетями.

Задание. Реализация логистической регрессии

Вам необходимо написать модуль на PyTorch, реализующий линейную модель для расчета $logits = XW + b$, где W и b – параметры (`nn.Parameter`) модели. Иначе говоря, здесь мы

реализуем своими руками модуль `nn.Linear` (в этом пункте его использование запрещено). Инициализируйте веса нормальным распределением (`torch.randn`).

```
class LinearRegression(nn.Module):
    def __init__(self, in_features: int, out_features: int, bias: bool = True):
        super().__init__()
        self.weights = nn.Parameter(torch.randn(in_features, out_features))
        self.bias = bias
        if bias:
            self.bias_term = nn.Parameter(torch.randn(out_features))

    def forward(self, x):
        x = torch.matmul(x, self.weights)
        if self.bias:
            x += self.bias_term
        return x
```

Обратите внимание, что мы предлагаем использовать `BCEWithLogitsLoss`, которая работает непосредственно с логитами, в отличие от `BCELoss`, куда мы передаём уже значения вероятностей (после операции `sigmoid`).

```
linear_regression = LinearRegression(in_features = 2, out_features = 1)
loss_function = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(linear_regression.parameters(), lr=0.05)
```

Вопрос 1. Сколько обучаемых параметров у получившейся модели? Имеется в виду суммарное количество отдельных числовых переменных, а не количество тензоров.

Ответ: Всего **3 обучаемых параметра** (2 веса и 1 биас). `requires_grad=True`.

Все параметры

```
print(list(linear_regression.parameters()))
```

```
[Parameter containing:
tensor([[0.3367],
        [0.1288]], requires_grad=True), Parameter containing:
tensor([0.2345], requires_grad=True)]
```

Beca

```
print(linear_regression.weights)
```

```
Parameter containing:
tensor([[0.3367],
        [0.1288]], requires_grad=True)
```

```
# Биас
print(linear_regression.bias_term)

Parameter containing:
tensor([0.2345], requires_grad=True)
```

Часть 1.3. Функции для работы с моделью (обучение и предсказание)

Задание. Функция обучения

Перейдём непосредственно к обучению.

Вот псевдокод, который поможет вам разобраться в том, что происходит во время обучения

```
for epoch in range(max_epochs): # <----- итерируемся по
дatasetу несколько раз
    for x_batch, y_batch in dataset: # <----- итерируемся по
datasetу. Так как мы используем SGD а не GD, то берем батчи заданного
размера
        optimizer.zero_grad() # <----- обнуляем градиенты
модели
        outp = model(x_batch) # <----- получаем "логиты" из
модели
        loss = loss_func(outp, y_batch) # <---- считаем "лосс" для
логистической регрессии
        loss.backward() # <----- считаем градиенты
optimizer.step() # <----- делаем шаг
градиентного спуска
        if convergence: # <----- в случае сходимости
выходим из цикла
            break
```

В коде ниже добавлено логирование `accuracy`.

Реализуем цикл обучения в виде функции, которая будет принимать в качестве параметров нашу модель, данные (созданный ранее DataLoader), оптимизатор и функцию потерь и проводить обучение (т.е. пересчитывать и обновлять веса модели) до выполнения определенных условий. Функция будет возвращать список из лоссов на всех батчах в цикле обучения и список из наборов всех значений параметров модели (`weights` и `bias_term`) по всем эпохам цикла обучения.

Значения параметров мы будем сохранять на случай, если модель переобучится и будут нужны не последние значения, а более старые. Можно было бы сохранять значения параметров не по всем, а по нескольким последним эпохам, но для простоты сохраним по всем эпохам.

Цикл обучения будет останавливаться при выполнении одного из двух условий:

- достижение максимального количества эпох
- замедление снижения валидационной метрики (например, чтобы модель не переобучилась).

Поскольку пока мы с вами еще не определились с метрикой, давайте следить за изменением `loss` (остановка цикла, если разность между усредненными значениями `loss` в текущей и предыдущей эпохах меньше определенного порога)

Максимальное количество эпох и порог, с которым будем сравнивать разницу усредненных значений `loss` между эпохами, мы также будем передавать в функцию в качестве параметров.

Комментарий

Следить и сравнивать изменение `train loss` может не всегда корректно отображать ситуацию с обучением. Например, иногда в диффузионных моделях при обучении функция потерь довольно быстро начинает колебаться в окрестности одного значения, однако качество генерации картинок еще долго может улучшаться. Однако в данном примере можно использовать прекращение изменения лосса как критерий остановки.

```
# выход по лоссу
def train_model(model, train_dataloader, optimizer, loss_func,
               max_epochs, tol = 1e-3):
    """
    args:
        model - модель для обучения
        train_dataloader - loader с обучающей выборкой
        optimizer - оптимизатор, взятый из модуля `torch.optim`
        loss_func - функция потерь, взятая из модуля `torch.nn`
        max_epochs - максимальное количество эпох обучения
        tol - порог, с которым будет сравниваться изменение loss,
        усредненного по эпохе

    # создадим переменные (на данный момент - пустые списки), куда будем
    # сохранять
    # лоссы по батчам, усредненные по эпохе лоссы и параметры модели
    batch_losses = []
    epoch_mean_losses = []
    model_param_dict = []

    for epoch in range(max_epochs):
        losses = [] # переменная losses - список, куда будем добавлять
        значения loss по каждому батчу текущей эпохи
        for it, (X_batch, y_batch) in enumerate(train_dataloader):
            # Далее далее по коду идут стандартные операции цикла
            обучения:
            optimizer.zero_grad()
```

```

    outp = model(X_batch)
    loss = loss_func(outp, y_batch)
    loss.backward()

    losses.append(loss.item())
    optimizer.step()

    # Для расчета accuracy вычисляем вероятности и предсказание
    # класса нашей моделью,
    # затем вычисляем среднее значение accuracy для текущего батча
    probabilities = torch.sigmoid(outp)
    preds = (probabilities > 0.5).type(torch.float32)
    batch_acc = (preds == y_batch).sum() / y_batch.size(0)

    # через каждые 100 батчей выводим среднее значение accuracy по
    # батчу
    if (epoch * len(train_dataloader) + it + 1) % 100 == 0:
        print(f"Iteration: {epoch * len(train_dataloader) + it +
1}\nBatch accuracy: {batch_acc}")

    batch_losses = batch_losses + losses    # добавляем список потерь
    # текущей эпохи к списку потерь по батчам всех предыдущих эпох
    epoch_mean_losses.append(sum(losses)/len(losses))    # вычисляем
    # средний по текущей эпохе лосс и добавляем к списку по предыдущим
    # эпохам
    model_param_dict.append(copy.deepcopy(model.state_dict()))    #
    # добавляем в список текущие значения параметров модели

    # Ниже реализован ранний выход из цикла обучения. Если разница
    # между средним по эпохе значением loss в текущей
    # и предыдущей эпохах меньше порога tol, то оператор return
    # завершит выполнение функции и обучение прекратится
    if epoch > 0 and abs(epoch_mean_losses[-1] - epoch_mean_losses[-
2]) < tol:
        print(f"\nIteration: {epoch * len(train_dataloader) + it
+1}. Convergence. Stopping iterations after {epoch + 1} epochs.")
        return batch_losses, model_param_dict

    return batch_losses, model_param_dict

```

Теперь давайте проверим нашу функцию. Запустим обучение с лимитом 100 эпох.

Замечание. Если вы захотите запустить обучение модели заново, не забудьте снова инициализировать модель и оптимайзер. Если этого не сделать, функция `train_model()` будет пытаться обновлять веса уже обученной ранее модели - то есть пытаться дообучить нашу модель. Если же мы сначала заново инициализируем модель, ее веса обновятся на случайные значения (см. строки `self.weights = nn.Parameter(...)` и `self.bias_term = nn.Parameter(...)` в функции инициализации `__init__()`) и обучение будет происходить с нуля.

#При повторном запуске функции train_model() заново инициализируйте модель и оптимайзер.

#Для этого раскомментируйте соответствующий код ниже:

```
#linear_regression = LinearRegression(2, 1)  
#optimizer = torch.optim.SGD(linear_regression.parameters(), lr=0.05)  
# linear_regression = LinearRegression(2, 1)  
# optimizer = torch.optim.SGD(linear_regression.parameters(), lr=0.05)
```

```
losses, model_param_dict = train_model(model = linear_regression,  
                                         train_dataloader =  
train_dataloader,  
                                         optimizer = optimizer,  
                                         loss_func = loss_function,  
                                         max_epochs=100,  
                                         tol = 1e-3  
                                         )
```

```
Iteration: 100  
Batch accuracy: 0.796875  
Iteration: 200  
Batch accuracy: 0.796875  
Iteration: 300  
Batch accuracy: 0.7890625  
Iteration: 400  
Batch accuracy: 0.84375  
Iteration: 500  
Batch accuracy: 0.875  
Iteration: 600  
Batch accuracy: 0.90625  
Iteration: 700  
Batch accuracy: 0.8671875  
Iteration: 800  
Batch accuracy: 0.8203125  
Iteration: 900  
Batch accuracy: 0.8203125  
Iteration: 1000  
Batch accuracy: 0.8671875  
Iteration: 1100  
Batch accuracy: 0.8828125  
Iteration: 1200  
Batch accuracy: 0.828125  
Iteration: 1300  
Batch accuracy: 0.875  
Iteration: 1400  
Batch accuracy: 0.875  
Iteration: 1500  
Batch accuracy: 0.8984375
```

Iteration: 1593. Convergence. Stopping iterations after 27 epochs.

Если вы захотите посмотреть на значения весов обученной модели, раскомментируйте код ниже

```
print('\nВариант 1 - вручную, в рамках работы с классами в ООП:')
print(linear_regression.weights)
print(linear_regression.bias_term)

print('\nВариант 2 - с помощью стандартного метода parameters() класса nn.Module')
print(list(linear_regression.parameters()))

print('\nВариант 3 - с помощью стандартного метода state_dict() класса nn.Module')
print(linear_regression.state_dict())
```

Вариант 1 - вручную, в рамках работы с классами в ООП:

```
Parameter containing:
tensor([[ 1.2288],
        [-3.7314]], requires_grad=True)
Parameter containing:
tensor([0.2708], requires_grad=True)
```

Вариант 2 - с помощью стандартного метода parameters() класса nn.Module

```
[Parameter containing:
tensor([[ 1.2288],
        [-3.7314]], requires_grad=True), Parameter containing:
tensor([0.2708], requires_grad=True)]
```

Вариант 3 - с помощью стандартного метода state_dict() класса nn.Module

```
OrderedDict({'weights': tensor([[ 1.2288],
                                [-3.7314]]), 'bias_term': tensor([0.2708])})
```

Замечание. Обратите внимание, при сохранении в список значений параметров модели после обучения на текущей эпохе, мы делали глубокую копию значения параметров:

```
model_param_dict.append(copy.deepcopy(model.state_dict()))
```

Если просто добавлять в список значения параметров:

`model_param_dict.append(model.state_dict())`, то при продолжении обучения и, соответственно, пересчете параметров, в списке `model_param_dict` будут изменяться значения всех элементов. Вы можете самостоятельно в этом убедиться, если уберете в функции обучения `copy.deepcopy()`, заново проведете обучение модели и выведете список всех параметров модели - значения не будут различаться между эпохами. Импорт модуля `copy` перед определением функции обучения был нужен для использования глубокого копирования.

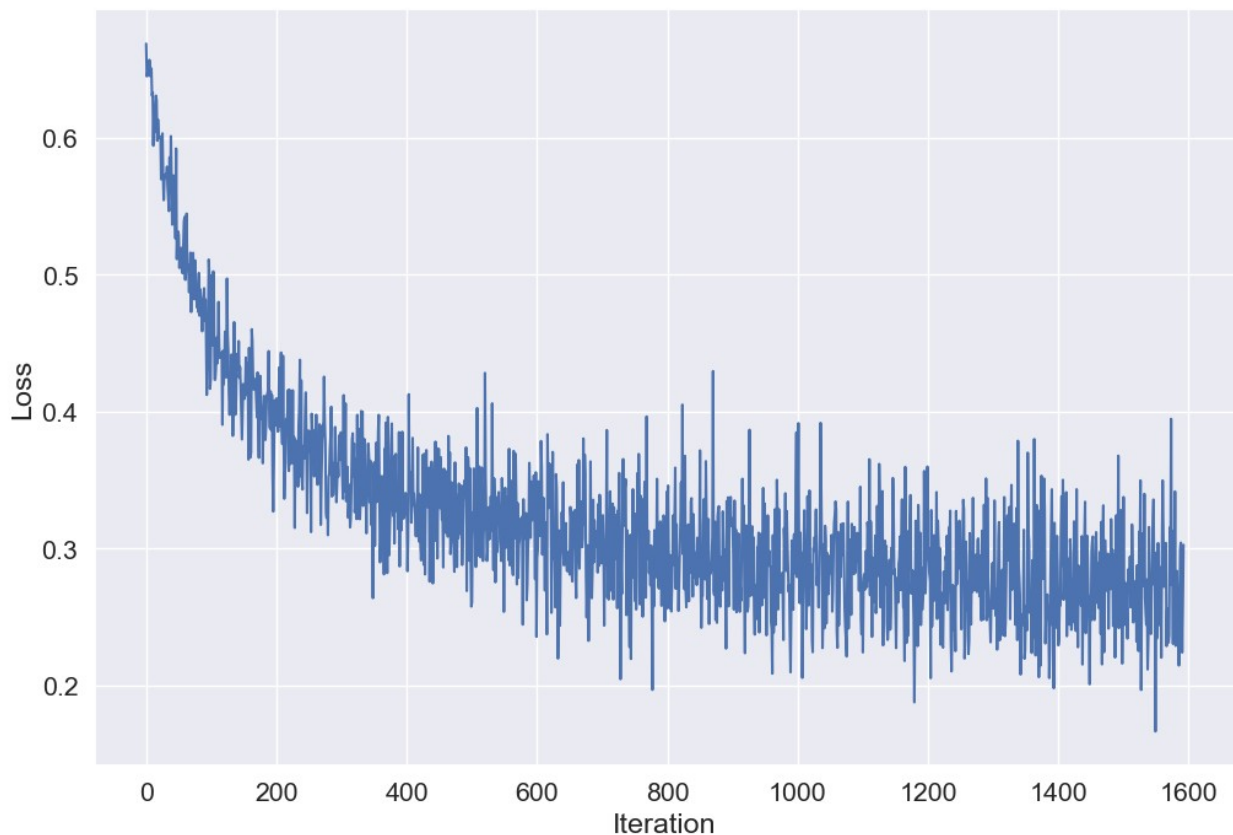
Это упражнение не является обязательной частью задания. Если вы его выполните, не забудьте вернуть в код `copy.deepcopy()` и обучить модель с корректной функцией.

Вопрос 2. Сколько итераций потребовалось, чтобы алгоритм сошелся? (ваш ответ может варьироваться в зависимости от запуска)

Ответ: 1593 или 1180 итераций в зависимости от запуска.

Визуализируем результаты

```
plt.figure(figsize=(12, 8))
plt.plot(range(len(losses)), losses)
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.show();
```



```
import numpy as np
sns.set_theme(style="white")
xx, yy = np.mgrid[-1.5:2.5:.01, -1.:1.5:.01]
grid = np.c_[xx.ravel(), yy.ravel()]
batch = torch.from_numpy(grid).type(torch.float32)
with torch.no_grad():
```

```

probs = torch.sigmoid(linear_regression(batch).reshape(xx.shape))
probs = probs.numpy().reshape(xx.shape)

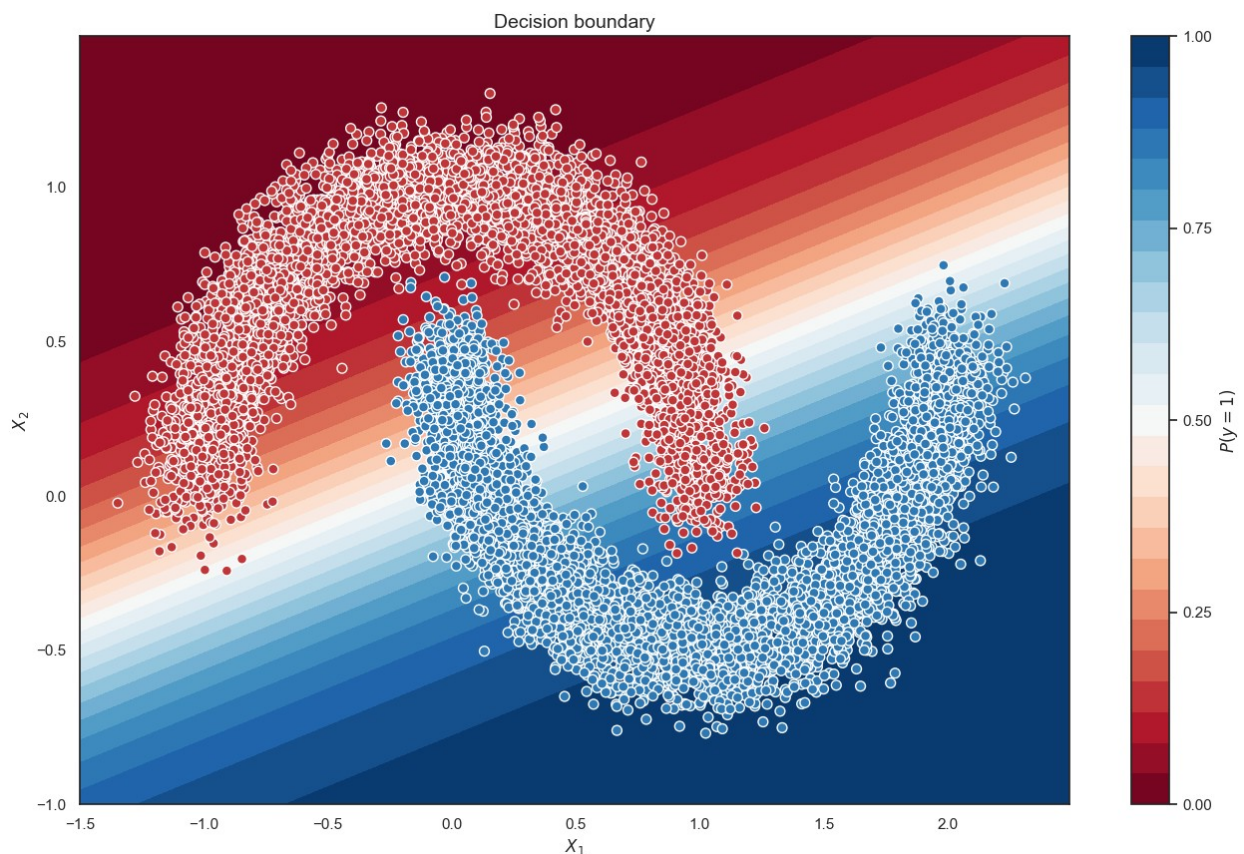
f, ax = plt.subplots(figsize=(16, 10))
ax.set_title("Decision boundary", fontsize=14)
contour = ax.contourf(xx, yy, probs, 25, cmap="RdBu",
                      vmin=0, vmax=1)
ax_c = f.colorbar(contour)
ax_c.set_label("$P(y = 1)$")
ax_c.set_ticks([0, .25, .5, .75, 1])

ax.scatter(X[100:, 0], X[100:, 1], c=y[100:], s=50,
           cmap="RdBu", vmin=-.2, vmax=1.2,
           edgecolor="white", linewidth=1)

ax.set_xlabel="$X_1$", ylabel="$X_2$")

plt.show();

```



Задание. Применим обученную модель к тестовым данным

Реализуйте функцию `predict` и посчитайте accuracy на `test`. Функция должна возвращать предсказание наиболее вероятного класса для каждого объекта из валидационной выборки:

```
@torch.no_grad()
def predict(dataloader, model):
    predictions = np.array([])
    for x_batch, _ in dataloader:
        outp = model(x_batch)
        probabilities = torch.sigmoid(outp)
        preds = (probabilities > 0.5).type(torch.float32)
        predictions = np.append(predictions, preds)

    return predictions
```

Сравните предсказания наиболее вероятного класса, полученные с помощью функции predict(), с фактическими классами в валидационной выборке и рассчитайте accuracy:

```
from sklearn.metrics import accuracy_score

y_pred = predict(val_dataloader, linear_regression)

accuracy = accuracy_score(y_val, y_pred)
print(f'Accuracy: {accuracy}')
```

Accuracy: 0.8852

Вопрос 3

Какое accuracy получается после обучения? (ваш ответ может варьироваться в зависимости от запуска)

Ответ: Accuracy = 0.8852.

Часть 2. Датасет MNIST

Часть 2.1. Подготовка данных

Датасет MNIST содержит рукописные цифры. Загрузим датасет и создадим DataLoader-ы. Пример можно найти в семинаре по полносвязным нейронным сетям.

Задание. Загрузчики данных

```
import os
from torchvision.datasets import MNIST
from torchvision import transforms as tfs

data_tfs = tfs.Compose([
    tfs.ToTensor(),
    tfs.Normalize((0.5), (0.5))
])
```

```
# install for train and test
root = './'
train_dataset_MNIST = MNIST(root, train=True, transform=data_tfs,
download=True)
val_dataset_MNIST = MNIST(root, train=False, transform=data_tfs,
download=True)

train_dataloader_MNIST = DataLoader(train_dataset_MNIST,
batch_size=128, shuffle=True)
valid_dataloader_MNIST = DataLoader(val_dataset_MNIST, batch_size=128,
shuffle=False)

# Далее будем использовать словарь из загрузчиков
loaders_MNIST = {"train": train_dataloader_MNIST, "valid":
valid_dataloader_MNIST}
```

Часть 2.2. Полносвязные нейронные сети

Сначала решим MNIST с помощью полносвязной нейронной сети.

Задание. Модель простой полносвязной нейронной сети

Создайте полносвязную нейронную сеть с помощью класса Sequential. Сеть состоит из:

- Уплотнения матрицы в вектор (nn.Flatten);
- Двух скрытых слоёв из 128 нейронов с активацией nn.ELU;
- Выходного слоя с 10 нейронами.

```
print(train_dataloader_MNIST.dataset[0][0].shape)
torch.Size([1, 28, 28])
activation = nn.ELU()

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28*28, 128),
    nn.ELU(),
    nn.Linear(128, 128),
    nn.ELU(),
    nn.Linear(128, 10)
).to(device)
```

Задайте лосс для обучения (кросс-энтропия).

Вспомним, что такое кросс-энтропийная функция потерь:

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^C y_{ni} \log(\hat{y}_{ni})$$

где:

- N — количество объектов в выборке,
- C — количество классов,
- y_{ni} — истинная метка (one-hot представление),
- $\hat{y}_{ni}$ — предсказанная вероятность модели для класса i на примере n .

Пример расчета

Рассмотрим пример расчета кросс-энтропийной функции потерь для задачи многоклассовой классификации с 10 классами. Предположим, что у нас есть один объект, истинный класс которого известен, а модель выдала свои предсказания в виде вероятностей для каждого класса.

Пусть объект является цифрой 2. Тогда:

$$y = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$$

Пусть модель выдала следующие вероятности для 10 классов:

$$\hat{y} = [0.1, 0.05, 0.6, 0.15, 0.05, 0.02, 0.01, 0.01, 0.01, 0.0]$$

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Только один элемент y_i равен 1, остальные равны 0. Поэтому:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i) = - y_2 \log(\hat{y}_2) = - 1 \log(0.6) \approx 0.51$$

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

device = "cuda" if torch.cuda.is_available() else "cpu"
print(device)

cuda
```

Задание. Функции для обучения и валидации модели

Давайте разберемся с кодом ниже, который подойдет для 90% задач в будущем.

```
for epoch in range(max_epochs): # <----- итерируемся по
    datasetу несколько раз
        model.train() # <----- переводим модель в режим
train
        for x_batch, y_batch in train_dataloader: # <--- итерируемся по
datasetу. Так как мы используем SGD а не GD, то берем батчи заданного
```



```

размера
    optimizer.zero_grad() # <----- обнуляем градиенты модели
    outp = model(x_batch) # <----- получаем "логиты" из
модели
    loss = criterion(outp, y_batch) # <-считаем "лосс" для
логистической регрессии
    loss.backward() # <----- считаем градиенты
    optimizer.step() # <----- делаем шаг градиентного
спуска
    count_metrics(outp, y_batch) # <----- считаем метрики на
train датасете

    model.eval() # <----- переводим модель в режим eval
    for x_batch, y_batch in val_dataloader: #<----- реализуем
режим test/eval
        with torch.no_grad(): # <----- НЕ считаем градиенты
            outp = model(x_batch) # <----- получаем "логиты"
из модели
            count_metrics(outp, y_batch) # <----- считаем метрики
на test датасете

```

Как и ранее обучение модели и проверку на валидационной выборке реализуем через функции. Создадим три функции:

1. для обучения модели на всей обучающей выборке (по всем батчам) на одной эпохе
2. для валидации модели на всей валидационной выборке (по всем батчам) на одной эпохе
3. для запуска функций 1. и 2. циклически по количеству эпох обучения модели

```

def fit_one_epoch(model, train_dataloader, optimizer, loss_func):
    """
    args:
        model - модель для обучения
        train_dataloader - loader с выборкой для обучения модели
        optimizer - оптимизатор, взятый из модуля `torch.optim`
        loss_func - функция потерь, взятая из модуля `torch.nn`

    функция возвращает метрику accuracy по эпохе на данных из
    train_dataloader
    """

    epoch_correct_predictions = 0.0
    epoch_all_predictions = 0.0
    model.train() # переводим модель в режим train

    for X_batch, y_batch in tqdm(train_dataloader):
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

```



```

# Далее далее по коду идут стандартные операции цикла обучения:
optimizer.zero_grad() # обнуляем градиенты модели

outp = model(X_batch) # Use model() to get outputs
loss = loss_func(outp, y_batch) # YOUR CODE. Compute loss

loss.backward() # считаем градиенты
optimizer.step() # делаем шаг градиентного спуска

# Рассчитаем ассурасу. Для этого определим количество верных
предсказаний
# и общее количество предсказаний в каждом батче, а затем
накопительным итогом посчитаем
# количество верных предсказаний и общее количество предсказаний
в рамках эпохи
preds = outp.argmax(-1)
batch_correct_predictions = (preds == y_batch).sum()
batch_all_predictions = len(preds)

epoch_correct_predictions += batch_correct_predictions
epoch_all_predictions += batch_all_predictions

epoch_accuracy = epoch_correct_predictions/epoch_all_predictions
return epoch_accuracy

# проверим работу функции
fit_one_epoch(model = model, train_dataloader =
train_dataloader_MNIST, optimizer = optimizer, loss_func = criterion)

{"model_id": "580a1cd940374077882f3c373ce7a8aa", "version_major": 2, "version_minor": 0}

tensor(0.8885, device='cuda:0')

def eval_one_epoch(model, val_dataloader):
    """
    args:
        model - модель для обучения
        val_dataloader - loader с валидационной/тестовой выборкой
    """

    epoch_correct_predictions = 0.0
    epoch_all_predictions = 0.0
    model.eval() # Переводим модель в режим eval

    for X_batch, y_batch in tqdm(val_dataloader):
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)
        # Далее далее по коду идут стандартные операции цикла валидации:
        with torch.no_grad():
            # Не забываем отключить расчет градиентов

```

```

        outp = model(X_batch) # Получаем "логиты" из модели
        preds = outp.argmax(-1) # Определяем наиболее вероятный класс
        для каждого объекта в батче

        # Рассчитаем accuracy
        batch_correct_predictions = (preds == y_batch).sum()
        batch_all_predictions = len(preds)

        epoch_correct_predictions += batch_correct_predictions
        epoch_all_predictions += batch_all_predictions

    epoch_accuracy = epoch_correct_predictions / epoch_all_predictions

    return epoch_accuracy

# проверим работу функции
eval_one_epoch(model = model, val_dataloader =
valid_dataloader_MNIST).item()

{"model_id": "b66dad3fe3904888a548c76e99a82476", "version_major": 2, "version_minor": 0}

0.9370999932289124

```

Ниже представлена функция, с помощью которой мы будем обучать модели. Эта функция вызывает `fit_one_epoch()` с числом раз, равным количеству эпох. Каждую эпоху также вызывается функция `eval_one_epoch()` для расчета ассурасу на валидационной выборке.

Функция возвращает словарь с ассурасу на обучающей и валидационной выборках на каждой эпохе.

```

def train_func(model, num_epochs, dataloaders, optimizer, loss_func):
    """
    args:
        model - модель для обучения
        num_epochs - количество эпох
        dataloaders - словарь loader'ов с обучающей и валидационной
        выборками
        optimizer - оптимизатор, взятый из модуля `torch.optim`
        loss_func - функция потерь, взятая из модуля `torch.nn`

    функция возвращает ассурасу на обучающей и валидационной выборках на
    каждой эпохе
    """

    accuracy = {"train": [], "valid": []}
    for epoch in range(num_epochs):
        print(f"\nEpoch: {epoch+1}")

        accuracy_train_one_epoch = fit_one_epoch(model = model,

```

```

train_dataloader =
dataloaders['train'],

optimizer = optimizer,
loss_func = loss_func)

print(f"Accuracy_train: {accuracy_train_one_epoch.item()}\n")

accuracy_test_one_epoch = eval_one_epoch(model = model,
                                          val_dataloader =
dataloaders['valid'])

print(f"Accuracy_test: {accuracy_test_one_epoch.item()}\n")

accuracy['train'].append(accuracy_train_one_epoch)
accuracy['valid'].append(accuracy_test_one_epoch)

return accuracy

```

Заново иницируем модель (для этого запустите код с инициацией в начале части 2.2. задания) и запустим обучение на 10 эпохах

```

optimizer = torch.optim.Adam(model.parameters())
max_epochs = 10
accuracy = train_func(model = model,
                      num_epochs = max_epochs,
                      dataloaders = loaders_MNIST,
                      optimizer = optimizer,
                      loss_func = criterion)

```

Epoch: 1

```

{"model_id": "3add8a880f354ac2855893ae2f9bc46e", "version_major": 2, "version_minor": 0}

```

Accuracy_train: 0.9484166502952576

```

{"model_id": "5dfea7fbc6ca46f8b04e190c065abbed", "version_major": 2, "version_minor": 0}

```

Accuracy_test: 0.9557999968528748

Epoch: 2

```

{"model_id": "f436775343664f11ab3e8a0c72dc5d33", "version_major": 2, "version_minor": 0}

```

Accuracy_train: 0.9634333252906799

```
{"model_id": "9e0dad10d36e4fcda829d90d07b15ed9", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9684000015258789

Epoch: 3

```
{"model_id": "8c3f35a3692f476b9db62c84f819feef", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9709666967391968

```
{"model_id": "d1ecda5f58d94dbba68c53797444323a", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9650999903678894

Epoch: 4

```
{"model_id": "7c96190bc4ef4d29a426ecf66e4ede02", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9762166738510132

```
{"model_id": "ec2e69a992f1457aa2134c18716491e6", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.972599983215332

Epoch: 5

```
{"model_id": "256a64ef724641c78e2ba2988295d072", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9809166789054871

```
{"model_id": "b427c6ec5f3246ae8f2e601296bbe26b", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9749999642372131

Epoch: 6

```
{"model_id": "f9afefdfef7304131899e4993d963247c", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9825833439826965

```
{"model_id": "7aca6ef15417408797d6114cec5f2345", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9741999506950378

Epoch: 7

```
{"model_id": "dc78996ce8954ef99f0fe5106837fd15", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.983833372592926

```
{"model_id": "3f442c1024b64d6a9208f5dd06f0cee2", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9752999544143677

Epoch: 8

```
{"model_id": "1ac392366be745ff85868a20fa00690f", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9869500398635864

```
{"model_id": "41c2a04b1d2443d3a37a8484c53a8426", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9736999869346619

Epoch: 9

```
{"model_id": "7f3e0b3f1e9e4541bce6ed2fa1d14e10", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9878333210945129

```
{"model_id": "464e3d72d5fd46a98b8f0d30c6cc6214", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9791999459266663

Epoch: 10

```
{"model_id": "b0499a03c6034f90a45b0b5503a5a008", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9876833558082581

```
{"model_id": "361c60ce61e2415abd1b3aa807b693ff", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9724999666213989

Функции активации

Функции активации добавляют **нелинейность в модель**, что позволяет сети решать сложные задачи, такие как классификация и регрессия. Без функций активации нейронная сеть была бы линейной моделью, так как **композиция линейных преобразований это линейное преобразование**.

ReLU (Rectified Linear Unit)

- Формула: $f(x) = \max(0, x)$
- Диапазон значений: $[0, \infty)$

Leaky ReLU

- Формула: $f(x) = \max(\alpha x, x)$, где α — маленькое положительное число (обычно $\alpha = 0.01$).
- Диапазон значений: $(-\infty, \infty)$

Exponential Linear Unit (ELU)

- Формула: $f(x) = \begin{cases} x & \text{если } x > 0 \\ \alpha (e^x - 1) & \text{если } x \leq 0 \end{cases}$
- Диапазон значений: $(-\alpha, \infty)$

Tanh (Гиперболический тангенс)

- Формула: $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Диапазон значений: $(-1, 1)$

Линейная функция активации

- Формула: $f(x) = x$
- Диапазон значений: $(-\infty, \infty)$

Обратите внимание, что данная функция активации **не добавляет** нелинейности.

Задание на дополнительный балл (3 балла)

Давайте попробуем закодировать наши функции активаций, а также потренируемся считать их градиенты.

Напишите классы для каждой из функций активации, а также для градиентов:

1. Leaky ReLU (1.5 балл: 0.5 forward pass, 1 -- backward (grad))
2. Tanh (1.5 балл: 0.5 forward pass, 1 -- backward (grad))

Для того, чтобы было понятнее, что нужно сделать -- приведём пример

```
class ReLU(nn.Module):
    def __init__(self, ):
        super().__init__()

    def forward(self, x):
        return torch.maximum(torch.zeros_like(x), x)

    def backward(self, x):
        grads = torch.zeros_like(x)
        grads[x > 0] = 1

        return grads

# test forward pass

torch_relu = nn.ReLU()
your_relu = ReLU()

x = torch.linspace(-3, 3, 100, requires_grad=True)

x_relu_custom = your_relu(x)
x_relu_torch = torch_relu(x)

assert torch.allclose(x_relu_custom, x_relu_torch)

grad_relu_custom = your_relu.backward(x)

x_relu_torch.sum().backward()
grad_relu_torch = x.grad.clone()
x.grad.zero_()

assert torch.allclose(grad_relu_custom, grad_relu_torch)

# Plot custom and torch functions
fig, axes = plt.subplots(1, 2, figsize=(20, 8))
fig.suptitle("Activation Functions Comparison")
```



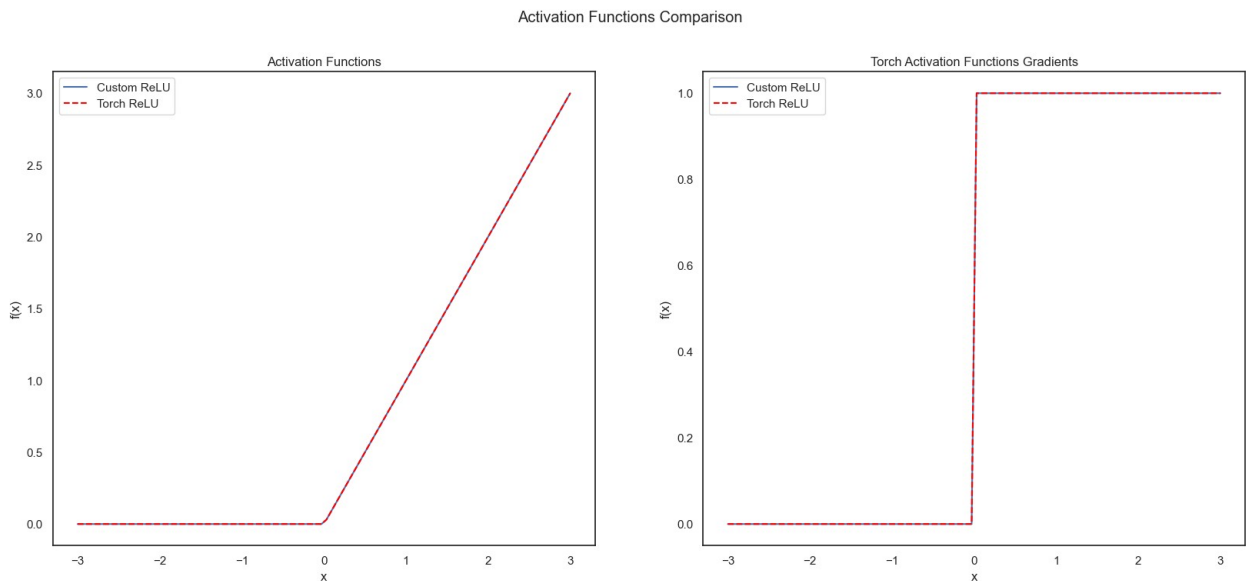
```

# Plot Custom Functions
axes[0].set_title("Activation Functions")
axes[0].plot(x.detach().numpy(), x_relu_custom.detach().numpy(),
label="Custom ReLU")
axes[0].plot(x.detach().numpy(), x_relu_torch.detach().numpy(),
color='red', label="Torch ReLU", linestyle='--')
axes[0].legend()
axes[0].set_xlabel("x")
axes[0].set_ylabel("f(x)")

# Plot Torch Functions
axes[1].set_title("Torch Activation Functions Gradients")
axes[1].plot(x.detach().numpy(), grad_relu_custom.detach().numpy(),
label="Custom ReLU")
axes[1].plot(x.detach().numpy(), grad_relu_torch.detach().numpy(),
color='red', label="Torch ReLU", linestyle='--')
axes[1].legend()
axes[1].set_xlabel("x")
axes[1].set_ylabel("f'(x)")

plt.show();

```



Теперь самостоятельно реализуйте две другие функции активаций.

```

class LeakyReLU(nn.Module):
    def __init__(self, alpha=0.01):
        super().__init__()
        self.alpha = alpha

    def forward(self, x):
        return torch.where(x > 0, x, self.alpha * x)

```

```

def backward(self, x):
    grads = torch.ones_like(x)
    grads[x < 0] = self.alpha
    return grads

class Tanh(nn.Module):
    def __init__(self, ):
        super().__init__()

    def forward(self, x):
        return torch.tanh(x)

    def backward(self, x):
        return 1 - torch.tanh(x)**2

# lets test your functions with torch functions

torch_leaky_relu = nn.LeakyReLU(negative_slope=0.01)
torch_tanh = nn.Tanh()

your_leaky_relu = LeakyReLU()
your_tanh = Tanh()

# Test forward pass
x_leaky_custom = your_leaky_relu(x)
x_leaky_torch = torch_leaky_relu(x)

assert torch.allclose(x_leaky_custom, x_leaky_torch)

x_tanh_custom = your_tanh(x)
x_tanh_torch = torch_tanh(x)

assert torch.allclose(x_tanh_custom, x_tanh_torch)

grad_leaky_custom = your_leaky_relu.backward(x)

x.grad.zero_()
x_leaky_torch.sum().backward()
grad_leaky_torch = x.grad.clone()

assert torch.allclose(grad_leaky_custom, grad_leaky_torch)

grad_tanh_custom = your_tanh.backward(x)

x.grad.zero_()
x_tanh_torch.sum().backward()

```

```

grad_tanh_torch = x.grad.clone()

assert torch.allclose(grad_tanh_custom, grad_tanh_torch)

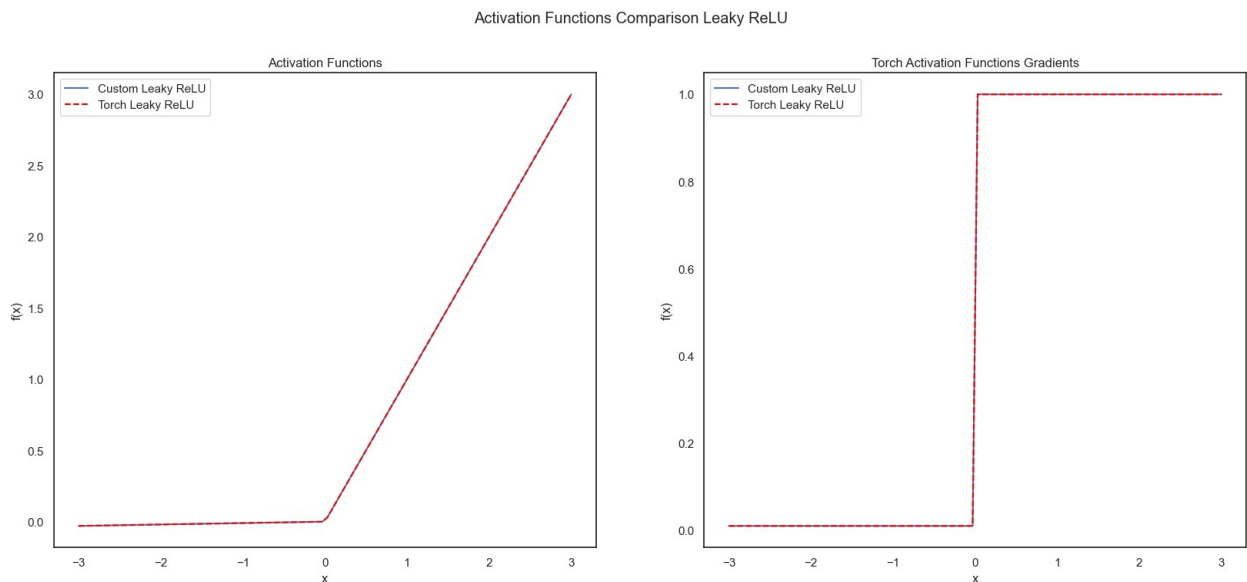
# Plot custom and torch functions
fig, axes = plt.subplots(1, 2, figsize=(20, 8))
fig.suptitle("Activation Functions Comparison Leaky ReLU")

# Plot Custom Functions
axes[0].set_title("Activation Functions")
axes[0].plot(x.detach().numpy(), x_leaky_custom.detach().numpy(),
label="Custom Leaky ReLU")
axes[0].plot(x.detach().numpy(), x_leaky_torch.detach().numpy(),
color='red', label="Torch Leaky ReLU", linestyle='--')
axes[0].legend()
axes[0].set_xlabel("x")
axes[0].set_ylabel("f(x)")

# Plot Torch Functions
axes[1].set_title("Torch Activation Functions Gradients")
axes[1].plot(x.detach().numpy(), grad_leaky_custom.detach().numpy(),
label="Custom Leaky ReLU")
axes[1].plot(x.detach().numpy(), grad_leaky_torch.detach().numpy(),
color='red', label="Torch Leaky ReLU", linestyle='--')
axes[1].legend()
axes[1].set_xlabel("x")
axes[1].set_ylabel("f'(x)")

plt.show();

```



```

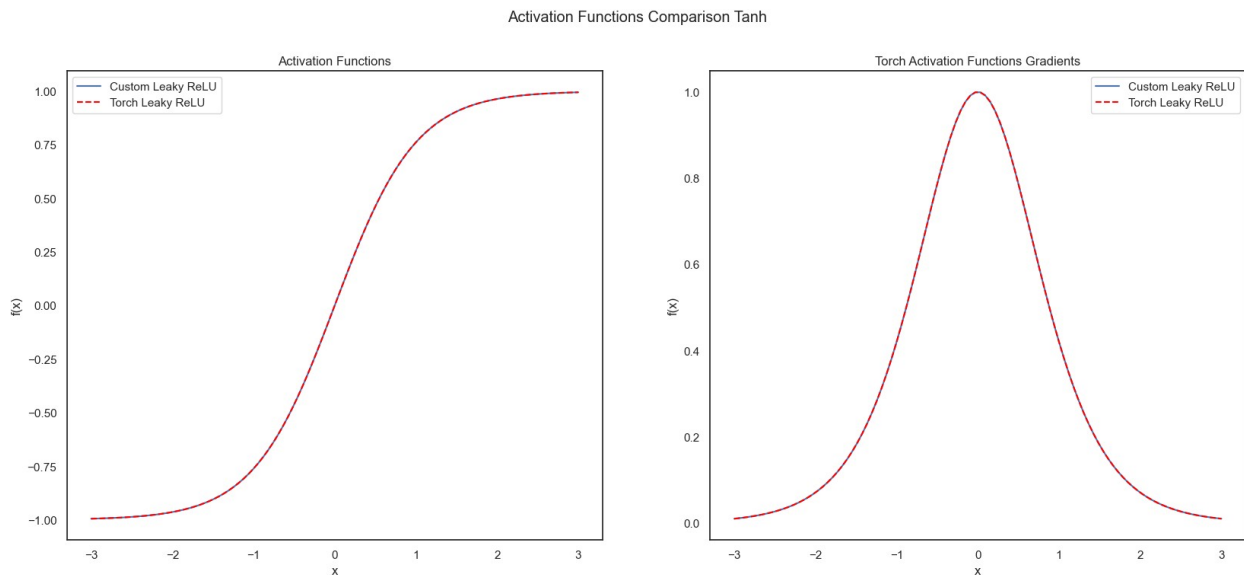
# Plot custom and torch functions
fig, axes = plt.subplots(1, 2, figsize=(20, 8))
fig.suptitle("Activation Functions Comparison Tanh")

# Plot Custom Functions
axes[0].set_title("Activation Functions")
axes[0].plot(x.detach().numpy(), x_tanh_custom.detach().numpy(),
label="Custom Leaky ReLU")
axes[0].plot(x.detach().numpy(), x_tanh_torch.detach().numpy(),
color='red', label="Torch Leaky ReLU", linestyle='--')
axes[0].legend()
axes[0].set_xlabel("x")
axes[0].set_ylabel("f(x)")

# Plot Torch Functions
axes[1].set_title("Torch Activation Functions Gradients")
axes[1].plot(x.detach().numpy(), grad_tanh_custom.detach().numpy(),
label="Custom Leaky ReLU")
axes[1].plot(x.detach().numpy(), grad_tanh_torch.detach().numpy(),
color='red', label="Torch Leaky ReLU", linestyle='--')
axes[1].legend()
axes[1].set_xlabel("x")
axes[1].set_ylabel("f'(x)")

plt.show();

```



Вопрос на подумать:

Какие недостатки есть у активаций **ReLU** и как эту проблему решает **Leaky ReLU**?

Что происходит с градиентами у активации **Tanh** и как это может помешать обучению?

Ответ: Недостатком функции активации ReLU является градиент равный 0 для отрицательных входных значений. Выход нейрона зануляется и не участвует в обучении, так как веса уже не обновляются. Функция активации Leaky ReLU решает эту проблему, так как она имеет небольшой наклон в отрицательной области и не зануляет градиенты входных значений. У функции активации Tanh при $|x| > 3$ градиенты зануляются, а при $2 > |x| > 3$ очень близки к 0. Веса нейронов перестают обновляться.

Задание. Протестируйте разные функции активации.

Попробуйте разные функции активации. Для каждой функции активации посчитайте массив validation accuracy. Лучше реализовать это в виде функции, берущей на вход активацию и получающей массив из accuracies.

```
# для функции активации ELU() воспользуемся полученной ранее оценкой accuracy
elu_accuracy = accuracy["valid"]
elu_accuracy

[tensor(0.9558, device='cuda:0'),
 tensor(0.9684, device='cuda:0'),
 tensor(0.9651, device='cuda:0'),
 tensor(0.9726, device='cuda:0'),
 tensor(0.9750, device='cuda:0'),
 tensor(0.9742, device='cuda:0'),
 tensor(0.9753, device='cuda:0'),
 tensor(0.9737, device='cuda:0'),
 tensor(0.9792, device='cuda:0'),
 tensor(0.9725, device='cuda:0')]
```

Оценку для остальных функций активации реализуем при помощи функции, которая будет инициализировать и обучать модель с необходимой функцией активации:

```
def test_activation_function(activation):
    model = nn.Sequential(
        nn.Flatten(),
        nn.Linear(28*28, 128),
        activation,
        nn.Linear(128, 128),
        activation,
        nn.Linear(128, 10)
    ).to(device)

    # optimizer = torch.optim.SGD(model.parameters())
    optimizer = torch.optim.Adam(model.parameters())
    accuracy = train_func(model=model,
                           num_epochs=max_epochs,
                           dataloaders=loaders_MNIST,
                           optimizer=optimizer,
                           loss_func=criterion)
```

```
        return accuracy
print('Identity')
plain_accuracy = test_activation_function(activation = nn.Identity())
['valid'] #There is no non-linearity
print('ReLU')
relu_accuracy = test_activation_function(nn.ReLU())['valid']
print('LeakyReLU')
leaky_relu_accuracy = test_activation_function(nn.LeakyReLU())
['valid']
print('Tanh')
tanh_accuracy = test_activation_function(nn.Tanh())['valid']

Identity
Epoch: 1

{"model_id": "3facc77afe2246f5b84cd256de7aeba7", "version_major": 2, "version_minor": 0}

Accuracy_train: 0.8715833425521851

{"model_id": "00c5f6334d004571b06c6220e59020fe", "version_major": 2, "version_minor": 0}

Accuracy_test: 0.899399995803833

Epoch: 2

{"model_id": "45ae89fa439b4a22a27b61365cc616bc", "version_major": 2, "version_minor": 0}

Accuracy_train: 0.9033333659172058

{"model_id": "d66a36afe3d04abda4008bba91d54532", "version_major": 2, "version_minor": 0}

Accuracy_test: 0.8933999538421631

Epoch: 3

{"model_id": "28830bb1049e4ea6bb27bbf6b9cb342a", "version_major": 2, "version_minor": 0}

Accuracy_train: 0.9070166945457458
```

```
{"model_id": "84a3b8d324084150855828cca37f3fe8", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9090999960899353

Epoch: 4

```
{"model_id": "abc42556d8654d84aa5a1e623ab1a13e", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9100333452224731

```
{"model_id": "137dd1d3fd4b4da9bfbdf0dab7aaf111", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9093999862670898

Epoch: 5

```
{"model_id": "0996a35e056d4c309b1bf9517d885803", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9108166694641113

```
{"model_id": "b72cd765128648308f5abcfdf35b548e", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9131999611854553

Epoch: 6

```
{"model_id": "0ef79933da764de78c583b7ffdf4b077", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9128833413124084

```
{"model_id": "a08731425ac14fe7902cd7042bca17e0", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9109999537467957

Epoch: 7

```
{"model_id": "0af59ea55cd940a8857ca8849bc74740", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9130499958992004

```
{"model_id": "a24fff5e160b4a718619c6elecc66f74", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.917199969291687

Epoch: 8

```
{"model_id": "3907cdf948fb4802bf0edf0ccb086130", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9150000214576721

```
{"model_id": "140646416ffa495e97e03b75eb6a71ac", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9114999771118164

Epoch: 9

```
{"model_id": "af2b24edd9e9445fac20b740d6dd610a", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9154500365257263

```
{"model_id": "df8f5acf6fa7494c88e9741169dc502f", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9172999858856201

Epoch: 10

```
{"model_id": "67cb528c9bdd438eb3cc4d01dd8dae32", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9157166481018066

```
{"model_id": "449aa0364c284049802cef881a023b3a", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.914900004863739

ReLU

Epoch: 1


```
{"model_id": "61899fa20bc7411f86e09ceb2cb5f319", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.8723999857902527

```
{"model_id": "26c4b0f82e1e4513809fad3fe547d3ce", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9281999468803406

Epoch: 2

```
{"model_id": "1d2246da065c40969af6d644364ed5d4", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9377333521842957

```
{"model_id": "c4bc3e7025ad4529892b9df212a8dbf5", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9454999566078186

Epoch: 3

```
{"model_id": "1ecc948304a2466ca572bd6051d0dde8", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9557333588600159

```
{"model_id": "8f9d416b9037408b8f9427eddf4449c6", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9566999673843384

Epoch: 4

```
{"model_id": "776da2f5f3434a5bbb04a206803a0041", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.964983344078064

```
{"model_id": "5819357b4bd34d4e9aea8c00ba07ddd3", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.965499997138977

Epoch: 5

```
{"model_id": "08e9c660a919431aa64e88c38e6a5b0c", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9702333211898804

```
{"model_id": "5d2bd8688a6748228a9718c1d6087f38", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9623000025749207

Epoch: 6

```
{"model_id": "a0b192d499f546499ec802346ba96995", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9742166996002197

```
{"model_id": "1a91e76b6d8147069bd7b40a1d0cfcaf", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9705999493598938

Epoch: 7

```
{"model_id": "3c07a0e878804bbbbc7c4cd087e642fd", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9762666821479797

```
{"model_id": "622f010b544346f7a34238b2b52bf13d", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9697999954223633

Epoch: 8

```
{"model_id": "c587e6a1fe624828ac9cf839fd4e6a0b", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9802666902542114

```
{"model_id": "4d8cfbe578e44524aa0a02e4a7fecfcd", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9669999480247498

Epoch: 9

{"model_id":"ff0c6d0d4f604918b2e116ba8f75b96a","version_major":2,"version_minor":0}

Accuracy_train: 0.9803500175476074

{"model_id":"ec3c2766eca746578ad17fea79504cea","version_major":2,"version_minor":0}

Accuracy_test: 0.9695000052452087

Epoch: 10

{"model_id":"1b9b8a5af09d4c018129d53fa6b8d6cf","version_major":2,"version_minor":0}

Accuracy_train: 0.9835333228111267

{"model_id":"c135b86415474df297ee942b28e65d17","version_major":2,"version_minor":0}

Accuracy_test: 0.9751999974250793

LeakyReLU

Epoch: 1

{"model_id":"7819f8ef3d8d4899a54b93e40bfe478e","version_major":2,"version_minor":0}

Accuracy_train: 0.8762833476066589

{"model_id":"3d0d8d685264406f9d084bd138e8fa41","version_major":2,"version_minor":0}

Accuracy_test: 0.9256999492645264

Epoch: 2

{"model_id":"6261c60775c5464e935e6c4b38929bab","version_major":2,"version_minor":0}

Accuracy_train: 0.944183349609375

```
{"model_id": "72e5672653504ca4b1d0c683515bc7ca", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9524999856948853

Epoch: 3

```
{"model_id": "51272b79c2a74f1aa8eaa9a345ebe750", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9596500396728516

```
{"model_id": "7185c7ab62bd48a6a3021fb0df2a4b69", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9620999693870544

Epoch: 4

```
{"model_id": "798dd29f87ef4b4d884e881307bfec65", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9687666893005371

```
{"model_id": "95a2d87721d04d708efbb4770201e34d", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9639999866485596

Epoch: 5

```
{"model_id": "3e30b9bda7e84c6ca88195a7eaeef3648", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9722499847412109

```
{"model_id": "5772a93281854a70899f8528b936e6ad", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9702999591827393

Epoch: 6

```
{"model_id": "e82011203184410087ef098b82570e17", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9764833450317383

```
{"model_id": "3e8acdfd21334a77a854ea5f036db6ad", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9638999700546265

Epoch: 7

```
{"model_id": "bcac07b3dfa34eb7bef68703bf166a17", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9789666533470154

```
{"model_id": "a1b624a0016f460da0c1e2b7a366292d", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9751999974250793

Epoch: 8

```
{"model_id": "03becd6cf73f4bb7a07d9f7240efbeff", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9807500243186951

```
{"model_id": "8dd362d4a2d54ba089dfd2715e624d93", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9708999991416931

Epoch: 9

```
{"model_id": "e39a4415d0614e7191f8900b0456d058", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9826833605766296

```
{"model_id": "1e5f280b20394f64bb5baaf41889316e", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.968999981880188

Epoch: 10

```
{"model_id": "5ced9588ab174532a75ece29243069f9", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9846000075340271

```
{"model_id": "348e6e3acff04053826174b4a916d680", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9741999506950378

Tanh

Epoch: 1

```
{"model_id": "855376a8d89b40bca29f2eb6d8502d44", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.8973667025566101

```
{"model_id": "ed647228bfc847abbeefabd3be7f6b83", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9393999576568604

Epoch: 2

```
{"model_id": "7bc6b866f6a94fca87c056c18f5f6969", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9534167051315308

```
{"model_id": "e75c7cf77fab45ba9fa932d04475472c", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9637999534606934

Epoch: 3

```
{"model_id": "35001c5d6d8c4e7590f09024da152865", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9650499820709229

```
{"model_id": "6ed2e6ef56834f0ca29f043b42f15c61", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9704999923706055

Epoch: 4

```
{"model_id": "e145b4ccae244a298221e063cb046a94", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.972100019454956

```
{"model_id": "df0bad148b914806923c4766e5d4a22c", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9692999720573425

Epoch: 5

```
{"model_id": "6ab879a838d7406588dd50606f3d0ae4", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.973800003528595

```
{"model_id": "ed188327f136492cb8f7b5c8210c92b7", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9711999893188477

Epoch: 6

```
{"model_id": "5b95840f57c3449d9aca151a594e53f8", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.977566659450531

```
{"model_id": "c0b17cb8e91d43abab9d50fa808fbb8b", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9690999984741211

Epoch: 7

```
{"model_id": "ec1becda76304ce6a84912b86f42781d", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9783833622932434

```
{"model_id": "ac6cdf5527e946f4a687e1b84dc076ac", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9678999781608582

Epoch: 8

```
{"model_id": "03998ca5ebe04962bclb9d3eea07fbae", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9803000092506409

```
{"model_id": "1d664239d5a940c992faee4456caa345", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9691999554634094

Epoch: 9

```
{"model_id": "145d9c539ff5452a9ad5df3550064e26", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9828667044639587

```
{"model_id": "94608e6ce3d74b2dac54a7e811d913fc", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9702000021934509

Epoch: 10

```
{"model_id": "226c117abf754ff3b34dd2bf07d81a3b", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9819333553314209

```
{"model_id": "d8e8da33704d44659c9c38a905f6d510", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9734999537467957

Визуализируем accuracy

Построим график accuracy/epoch для каждой функции активации.

```
sns.set_theme(style="darkgrid", font_scale=1.4)
```

```
plt.figure(figsize=(16, 10))
```

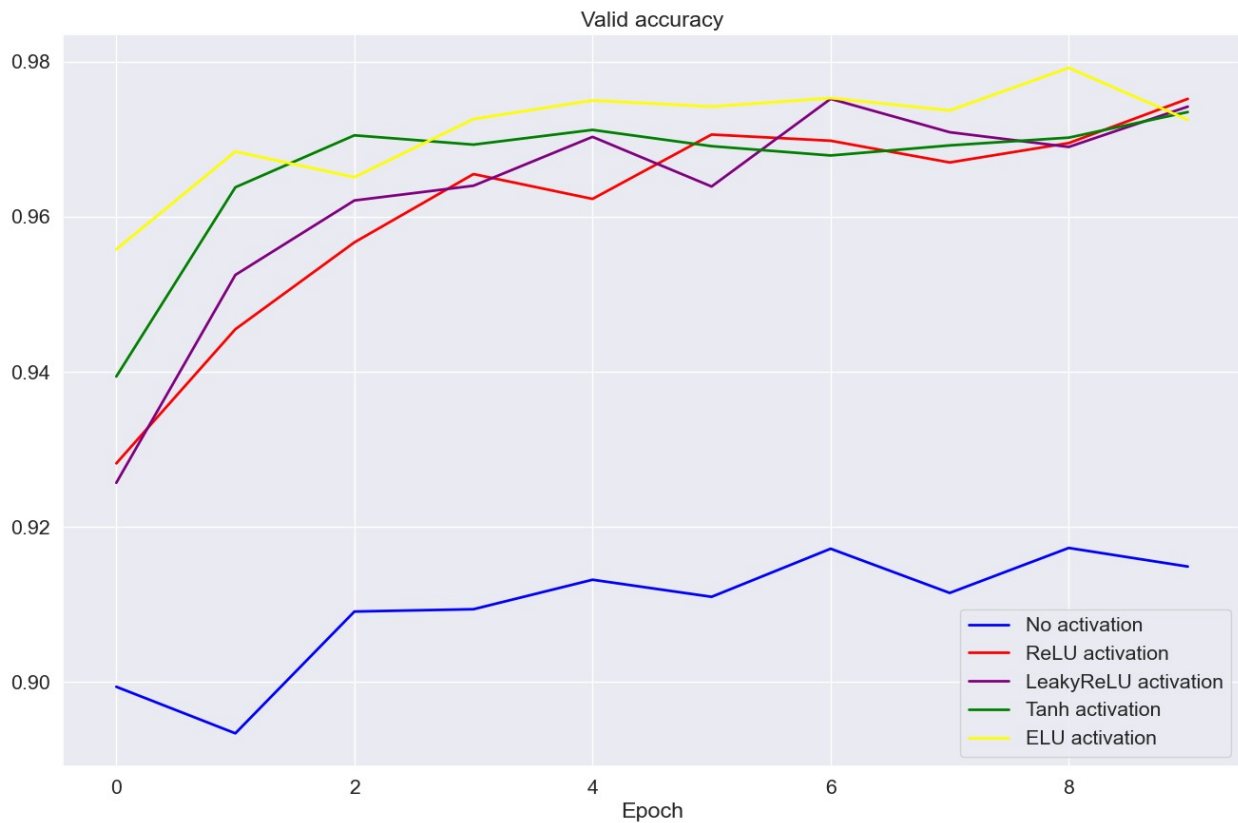


```

plt.title("Valid accuracy")
plt.plot(range(max_epochs), [t.cpu().numpy() for t in plain_accuracy],
label="No activation", linewidth=2, color = 'blue')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in relu_accuracy],
label="ReLU activation", linewidth=2, color = 'red')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in
leaky_relu_accuracy], label="LeakyReLU activation", linewidth=2, color
= 'purple')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in tanh_accuracy],
label="Tanh activation", linewidth=2, color = 'green')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in elu_accuracy],
label="ELU activation", linewidth=2, color = 'yellow')
plt.legend()
plt.xlabel("Epoch")

plt.show();

```



Вопрос 4. Какая из активаций показала наивысший **accuracy** к концу обучения?

Ответ: Максимальная точность на всех эпохах показала ELU, на последней эпохе - ReLU.

Часть 2.3. Сверточные нейронные сети

Задание. Ядра свёртки

Сначала немного поработаем с самим понятием ядра свёртки.

```
import numpy as np

import seaborn as sns
from matplotlib import pyplot as plt

import torch
from torch import nn
from torch.nn import functional as F

# !wget https://img.the-village.kz/the-village.com.kz/post-cover/5x5-
# I6oiwjmq79dMCZMEbA-default.jpg -O sample_photo.jpg

import cv2
sns.set_theme(style="white")
img = cv2.imread("sample_photo.jpg")
RGB_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(12, 8))
plt.imshow(RGB_img)

plt.show();
```



Попробуйте посмотреть как различные свертки влияют на фото. Например, попробуйте

А)

```
[0, 0, 0],
[0, 1, 0],
[0, 0, 0]
```

Б)

```
[0, 1, 0],
[0, -2, 0],
[0, 1, 0]
```

В)

```
[0, 0, 0],
[1, -2, 1],
[0, 0, 0]
```

Г)

```
[0, 1, 0],  
[1, -4, 1],  
[0, 1, 0]
```

Д)

```
[0, -1, 0],  
[-1, 5, -1],  
[0, -1, 0]
```

Е)

```
[0.0625, 0.125, 0.0625],  
[0.125, 0.25, 0.125],  
[0.0625, 0.125, 0.0625]
```

Не стесняйтесь пробовать свои варианты!

```
conv_1 = np.array([[0, 0, 0], [0, 1, 0], [0, 0, 0]]) # A  
conv_2 = np.array([[0, 1, 0], [0, -2, 0], [0, 1, 0]]) # Б  
conv_3 = np.array([[0, 0, 0], [1, -2, 1], [0, 0, 0]]) # В  
conv_4 = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]]) # Г  
conv_5 = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]]) # Д  
conv_6 = np.array([[0.0625, 0.125, 0.0625], [0.125, 0.25, 0.125],  
[0.0625, 0.125, 0.0625]]) # Е  
  
def show_conv_transformation(image, convolution):  
    img_t = torch.from_numpy(image).type(torch.float32).unsqueeze(0)  
    img_t = img_t.permute(0, 3, 1, 2) # [BS, H, W, C] -> [BS, C, H, W]  
    img_t = nn.ReflectionPad2d(1)(img_t) # Pad Image for same output  
    size  
  
    kernel = torch.tensor(convolution).reshape(1, 1, 3,  
3).type(torch.float32)  
    kernel = kernel.repeat(3, 3, 1, 1)  
  
    result = F.conv2d(img_t, kernel)[0]  
    result = torch.clip(input = result, min = 0, max = 256*3)  
  
    plt.figure(figsize=(5, 3))  
    result_np = result.permute(1, 2, 0).numpy() / 256 / 3  
    plt.imshow(result_np)  
    plt.show()  
  
    return None
```

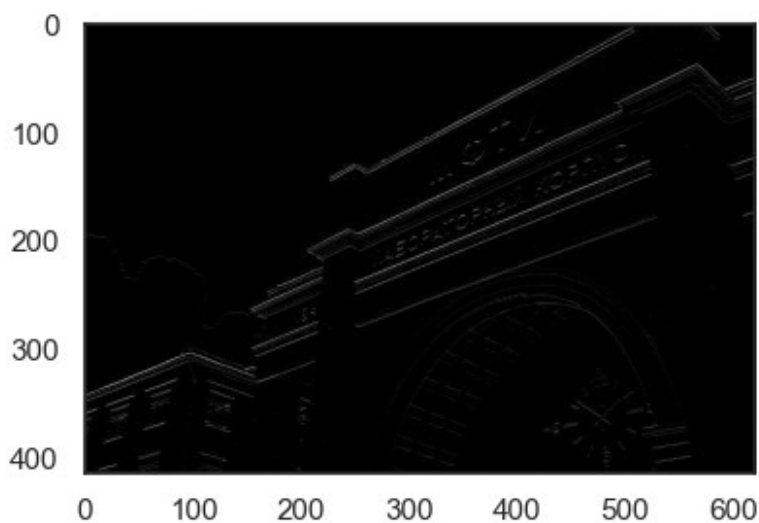
Посмотрите на работу разных сверток при помощи функции show_conv_transformation():

```
for conv, name in zip([conv_1, conv_2, conv_3, conv_4, conv_5,
conv_6], 'АБВГДЕ'):
    print(name)
    show_conv_transformation(image = RGB_img, convolution = conv)
```

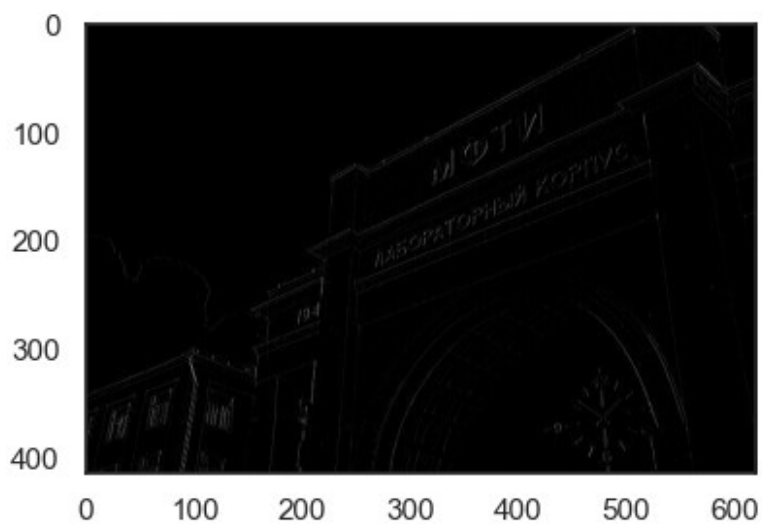
А



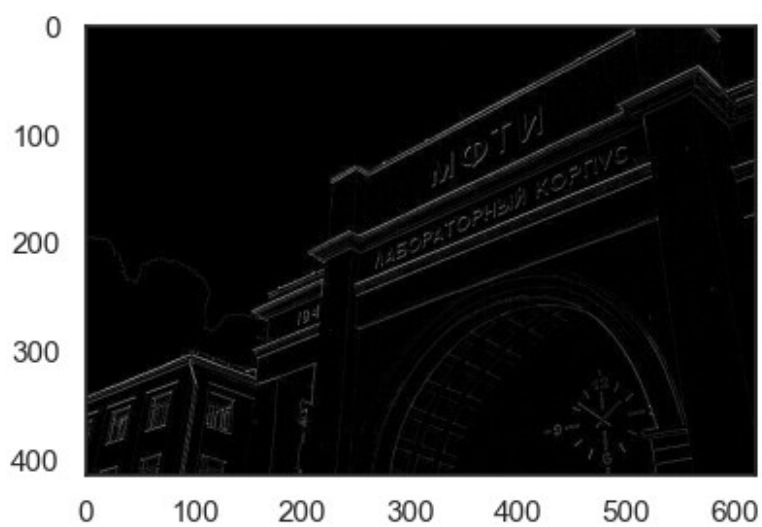
Б



В



Г



Д



Е



Вопрос 5. Как можно описать действия ядер, приведенных выше? Сопоставьте для каждой буквы число.

- 1) Размытие
- 2) Увеличение резкости
- 3) Тожественное преобразование
- 4) Выделение вертикальных границ
- 5) Выделение горизонтальных границ
- 6) Выделение границ

Ответ: Ядра выделяют определенные паттерны.

- А - 3 (Тождественное преобразование)
- Б - 5 (Выделение горизонтальных границ)
- В - 4 (Выделение вертикальных границ)
- Г - 6 (Выделение границ)
- Д - 2 (Увеличение резкости)
- Е - 1 (Размытие)

Задание. Реализуйте класс модели LeNet

Если мы сделаем параметры сверток обучаемыми, то можем добиться хороших результатов для задач компьютерного зрения. Реализуйте архитектуру LeNet, предложенную еще в 1998 году! На этот раз используйте модульную структуру (без помощи класса Sequential).

Наша нейронная сеть будет состоять из

- Свёртки 3x3 (1 карта на входе, 6 на выходе) с активацией ReLU;
- MaxPooling-а 2x2;
- Свёртки 3x3 (6 карт на входе, 16 на выходе) с активацией ReLU;
- MaxPooling-а 2x2;
- Уплотнения (nn.Flatten);
- Полносвязного слоя со 120 нейронами и активацией ReLU;
- Полносвязного слоя с 84 нейронами и активацией ReLU;
- Выходного слоя из 10 нейронов.

```
print(train_dataloader_MNIST.dataset[0][0].shape)
torch.Size([1, 28, 28])

class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        # 1 input image channel, 6 output channels, 3x3 square conv
        kernel
        # 28 x 28 : (28 - kernel_size(3) + padding(0)) // stride(1) +
        1 = (28 - 3 + 0) // 1 + 1 = 26      conv1
        self.conv1 = nn.Conv2d(in_channels = 1, out_channels = 6,
        kernel_size = 3)
        # 26 x 26 : 26 // 2 = 13
        pool1
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        # 13 x 13 : (13 - 3 + 0) // 1 + 1 = 11
        conv2
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16,
        kernel_size=3)
        # 11 x 11 : 11 // 2 = 5
        pool2
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
```



```

        # out_channels(16) * 5 * 5
    fl

    self.fl = nn.Flatten()
    self.fc1 = nn.Linear(16*5*5, 120)
    self.fc2 = nn.Linear(120, 84)
    self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool1(F.relu(self.conv1(x)))
        x = self.pool2(F.relu(self.conv2(x)))
        x = self.fl(x)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x

device = "cuda" if torch.cuda.is_available() else "cpu"
model_LeNet = LeNet().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_LeNet.parameters())

loaders_MNIST = {"train": train_dataloader_MNIST, "valid":
valid_dataloader_MNIST}

```

Задание. Обучите CNN-модель

Используйте функцию для обучения модели из части задания про полносвязные нейронные сети.

В качестве возможных улучшений рекомендуем добавить туда:

1. Валидацию каждые n шагов
2. Добавить сохранение промежуточных весов. Может так получиться, что с какого-то момента модель начинает переобучаться и использование весов с более ранних шагов даёт более качественный результат. Именно поэтому полезно сохранять веса не только после последней эпохи, но и с промежуточных этапов обучения.

```

max_epochs = 10

accuracy = train_func(model=model_LeNet,
                      num_epochs=max_epochs,
                      dataloaders=loaders_MNIST,
                      optimizer=optimizer,
                      loss_func=criterion)

```

Epoch: 1

```
{"model_id": "0f724c715f80468fa40edfba6d309649", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.8755500316619873

```
{"model_id": "fe7dcd97d4504078ba8c6aea1c05cad4", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.965999960899353

Epoch: 2

```
{"model_id": "1af1a18e755a4163a0d954c5cab2b7f5", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9713667035102844

```
{"model_id": "b1a91daec8fb4cf096ec70b9a4340beb", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9794999957084656

Epoch: 3

```
{"model_id": "45275e6aa5054da1a15050bb5f637405", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9795500040054321

```
{"model_id": "25c75b7aadcb41e688815ef1243479b9", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9824999570846558

Epoch: 4

```
{"model_id": "934c90c0d8f245169197b904f919d8af", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9833166599273682

```
{"model_id": "3bcfa337b24d4f70a5d051b64222f8f6", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9835000038146973

Epoch: 5

```
{"model_id": "e794153f3d6f4c038928dda136e9a480", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9865333437919617

```
{"model_id": "5e15f50791d144248ff469e0c9e43c2b", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9869999885559082

Epoch: 6

```
{"model_id": "01f1fd484aba46d9a0f3dd85e516b582", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9888333678245544

```
{"model_id": "eda7f24c026e42079af698878fcf5962", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9883999824523926

Epoch: 7

```
{"model_id": "c45243566a27413185fe08719f948a1e", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9898499846458435

```
{"model_id": "c07b61c592994745be18613440313cc6", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9886999726295471

Epoch: 8

```
{"model_id": "e608840730a341ee9e36e17738f4f33f", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9917833209037781

```
{"model_id": "6c39922fbbaa43f987239eefc37d31aa", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9893999695777893

Epoch: 9

```
{"model_id": "f13aaf359575468f879d489aec2f9983", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9927000403404236

```
{"model_id": "1f2fbffcab3f40d3ab04e3a3cb179801", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9879999756813049

Epoch: 10

```
{"model_id": "63dd0e49135948e0945936dffc6c8beb", "version_major": 2, "version_minor": 0}
```

Accuracy_train: 0.9936000108718872

```
{"model_id": "b5034a77bac3474196e2226ecc80bbab", "version_major": 2, "version_minor": 0}
```

Accuracy_test: 0.9884999990463257

lenet_accuracy = accuracy["valid"]

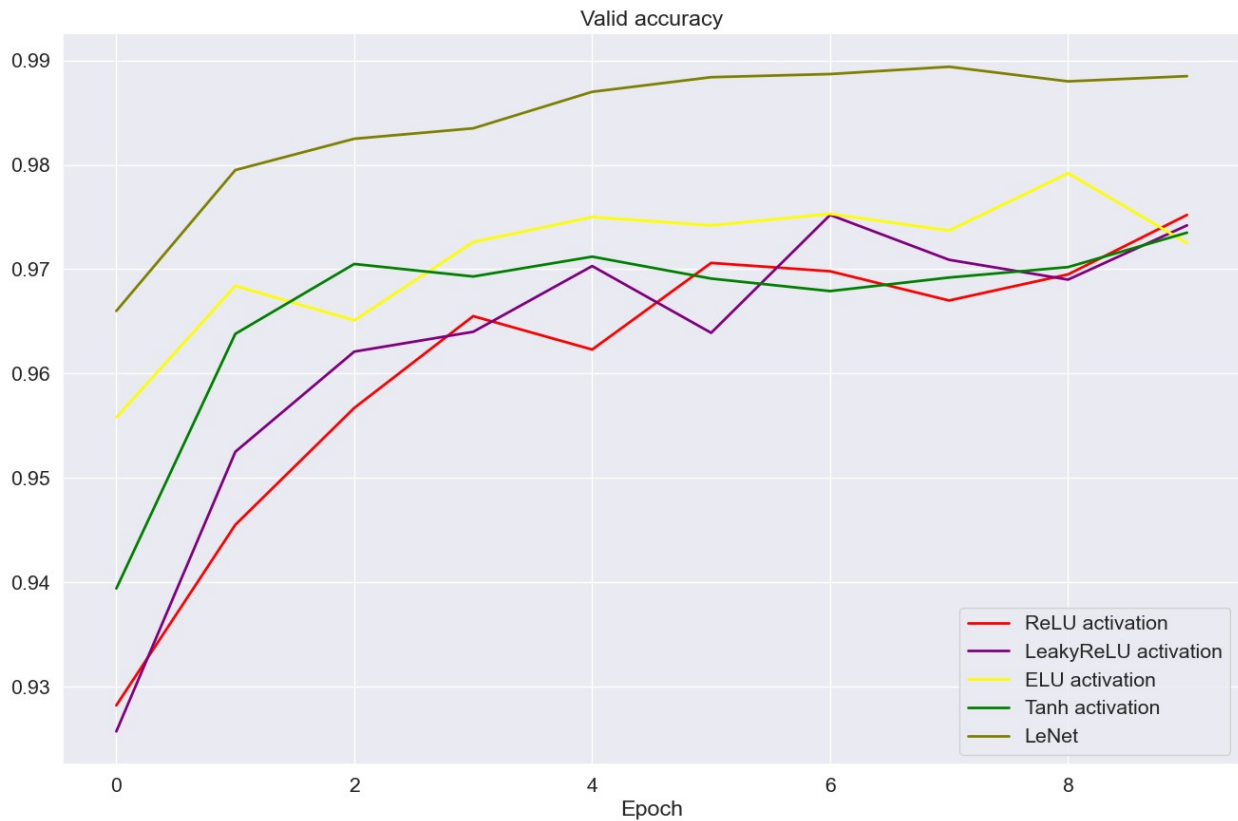
Сравним с предыдущем пунктом

```
sns.set_theme(style="darkgrid", font_scale=1.4)
```

```
plt.figure(figsize=(16, 10))
plt.title("Valid accuracy")
plt.plot(range(max_epochs), [t.cpu().numpy() for t in relu_accuracy],
label="ReLU activation", linewidth=2, color = 'red')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in
leaky_relu_accuracy], label="LeakyReLU activation", linewidth=2, color
= 'purple')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in elu_accuracy],
label="ELU activation", linewidth=2, color = 'yellow')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in tanh_accuracy],
label="Tanh activation", linewidth=2, color = 'green')
plt.plot(range(max_epochs), [t.cpu().numpy() for t in lenet_accuracy],
```

```
label="LeNet", linewidth=2, color = 'olive')
plt.legend()
plt.xlabel("Epoch")

plt.show();
```



```
accuracy = [t.cpu().numpy() for t in lenet_accuracy][-1]
print(f'Accuracy: {accuracy:.2f}')
```

Accuracy: 0.99

Вопрос 6 Какое **ассигасу** получается после обучения с точностью до двух знаков после запятой?

Ответ: Accuracy = 0.99.